pg_project 설치 가이드 (아키텍처 포함)

대상   : naturalTuna/pg_project (PR #5, #6 머지 기준)
OS   : RHEL 계열 8/9 (Rocky, Alma), x86_64
작성일   : 2026-11-08
같이 보기 : 운영가이드.txt (설치 후 관리 / 장애 대응)

목차

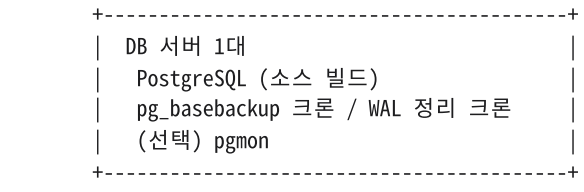
1. 구성 요소
2. 아키텍처
3. 사전 준비 (체크리스트)
4. [1단계] 패키지 다운로드 - 인터넷 서버
5. [2단계] 폐쇄망 반입
6. [3단계] single 설치
7. [3단계] HA 설치
8. 무인 설치 (응답 파일)
9. 설치 후 확인
10. HA 동작 테스트 시나리오
11. 설치 문제 해결

1. 구성 요소

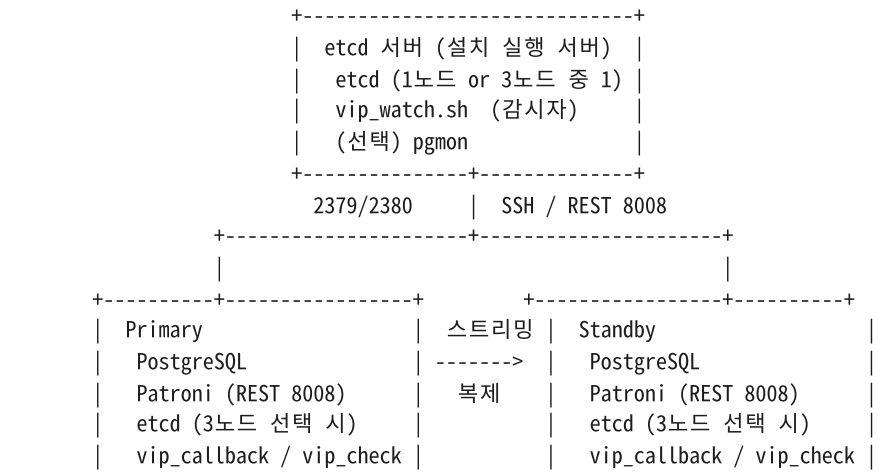
pg_download.sh	[인터넷 서버] PG 소스, 의존 rpm, extension, etcd, Patroni wheel, pgmon, pg_tune 을 postgresql/ 로 패키징
postgresql_install.sh	[폐쇄망 서버] 소스 빌드 -> initdb -> 튜닝 -> 백업 크론
lib/common.sh	출력, conf 생성, rpm 설치, PG/extension 빌드
lib/input.sh	입력 / 응답 파일 / 확인 화면
lib/backup.sh	백업, WAL 정리, PITR 복구 스크립트 생성
lib/vip.sh	VIP (callback + 보정 cron + etcd 서버 감시자)
lib/ha_safety.sh	watchdog / failsafe_mode / synchronous_mode
lib/single.sh, ha.sh	모드별 설치
pgmon	터미널 모니터링 (수집 데몬 + 알림)
pg_tune	규칙 기반 쿼리 튜너 (hypopg / pg_hint_plan 검증, pgmon 연동)

2. 아키텍처

2-1. single 모드



2-2. HA 모드 (3대)



```
|   softdog (watchdog)       |           |   softdog (watchdog)       |
|   [VIP] <- 앱 접속         |           |                            |
+----------------------------+           +----------------------------+
```

- etcd : 누가 Primary 인지(leader key) 저장. 3노드면 1대 장애에도 동작
- Patroni : PG 기동/정지/승격/복제 구성을 자동 관리
- VIP : Primary 를 따라 이동. 앱은 VIP 하나로 접속

2-3. 장애 시 흐름

```
[Primary 서버 다운]
  Patroni 가 leader key 갱신 못 함 -> ttl(30초) 만료
  -> Standby Patroni 가 승격 (maximum_lag_on_failover 이내일 때만)
  -> Standby 의 callback(on_role_change primary) 이 VIP 추가 + arping
  -> 앱 재접속 시 같은 VIP 로 새 Primary 접속
  예상 전환 시간 : 약 30~40초

[Primary 네트워크만 단절]
  Primary 는 etcd 접근 불가 -> 스스로 demote -> callback 이 VIP 제거
  반대편은 위와 같이 승격 + VIP 추가

[Primary 노드 freeze (커널 hang, VM 정지, Patroni 멈춤)]
  스크립트가 못 돎 -> watchdog(softdog) 이 노드를 리부트 (펜싱)

[etcd 전체 장애]
  failsafe_mode=true : Primary 가 모든 멤버와 REST 통신되면 Primary 유지
  failsafe_mode=false: Primary 도 demote -> 쓰기 중단
  어느 경우든 이 동안 failover 는 일어나지 않음
```

2-4. VIP 3중 관리

```
1) vip_callback.sh  Patroni callback, 역할 변경 즉시 add/del     (각 DB 노드)
2) vip_check.sh     crontab 매분, 20초 간격 REST /primary 로 보정 (각 DB 노드)
3) vip_watch.sh     crontab 매분, 리더/VIP 위치 교차 확인 + 알림 (etcd 서버)
etcd 서버가 죽어도 1), 2) 는 계속 동작
```

2-5. 설치 후 디렉토리 (기본값 기준)

```
/postgres/app                 엔진 (bin, lib, share)
/postgres/app/etc/patroni     patroni.yml, vip_callback.sh, vip_check.sh   (HA)
/postgres/app/etc/backup      pg_basebackup.sh, wal_cleanup.sh, pitr_restore.sh
/postgres/app/etc/configure_options.txt   빌드 시 configure 옵션
/pgdata/data                  데이터 디렉토리
/pgdata/log                   PG 로그, vip.log, wal_cleanup.log
/pgbackup/arc                 WAL 아카이브
/pgbackup/back                베이스 백업
~/.postgresrc                 환경변수 + alias (.bashrc 에서 로드)
~/pgvip/vip_watch.sh          VIP 감시자 (etcd 서버)
/etc/etcd/etcd.conf           etcd 설정 (systemd: etcd)
/etc/systemd/system/patroni.service
```

2-6. 포트 (방화벽 허용 필요)

출발 -> 도착	포트	용도
앱 -> VIP / Primary / Standby	5432	DB 접속 (입력한 포트)
Primary <-> Standby	5432	복제, pg_rewind
Primary/Standby -> etcd 노드	2379	Patroni -> etcd
etcd 노드 <-> etcd 노드	2380	etcd 피어 (3노드)
etcd 서버 -> Primary/Standby	8008	Patroni REST (vip_watch)
Primary <-> Standby	8008	Patroni REST (failsafe)
etcd 서버 -> Primary/Standby	22	SSH (설치, vip_watch)
Primary <-> Standby	22	SSH (설치)

3. 사전 준비 (체크리스트)

```
[공통]
[ ] 다운로드 서버 = 설치 서버와 같은 OS / 메이저 버전 (rpm, python 버전 일치)
[ ] 실행 계정 생성 (예: postgres)  -  설치는 이 계정으로 실행
```

```
 [ ] 실행 계정 sudo NOPASSWD  (rpm 설치, systemd 등록)
       echo 'postgres ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/postgres
       chmod 440 /etc/sudoers.d/postgres
 [ ] 디스크: 엔진(/postgres), 데이터(/pgdata), 백업/아카이브(/pgbackup) 마운트
 [ ] 최상위 디렉토리를 실행 계정 소유로 미리 생성 (스크립트는 sudo 없이 mkdir 함)
       sudo mkdir -p /postgres /pgdata /pgbackup
       sudo chown -R postgres:postgres /postgres /pgdata /pgbackup
       HA 는 etcd 서버 포함 3대 모두 (etcd 서버에서 빌드 + 로그 기록)
 [ ] ko_KR.utf8 로케일 존재 확인 (locale -a | grep -i ko_KR)

[HA 추가]
 [ ] 세 서버 /etc/hosts 에 etcd / primary / standby 등록 (모든 서버 동일하게)
 [ ] SSH 키 교환 (패스워드 없이)
       etcd 서버 -> primary, standby
       primary  <-> standby
       확인: ssh -o BatchMode=yes <호스트> hostname
 [ ] 2-6 의 포트 방화벽 허용
 [ ] 시간 동기화 (chronyd) - etcd / Patroni 는 서버 간 시간이 맞아야 안정적
 [ ] (VIP 사용 시)
       같은 L2 서브넷의 미사용 IP 1개 확보  (ping 해서 응답 없어야 함)
       DB 노드에 iproute, iputils(arping), curl 설치 확인
       클라우드(AWS/GCP 등)는 ARP 기반 VIP 불가 -> LB/DNS 사용
 [ ] (watchdog 사용 시)
       DB 노드에 다른 watchdog 데몬(watchdog, wdmd 등)이 /dev/watchdog 를
       쓰고 있지 않은지:  ls -l /dev/watchdog*; sudo lsof /dev/watchdog
```

4. [1단계] 패키지 다운로드 - 인터넷 서버

```
$ git clone https://github.com/naturalTuna/pg_project.git
$ cd pg_project
$ bash pg_download.sh                # 기본
$ bash pg_download.sh --verify       # 시험 빌드까지 (빌드 환경 설치됨, 수 분 추가)
$ bash pg_download.sh --lock BUILD_LOCK.txt   # 예전과 같은 구성으로 다시 받기

질문 항목
  PostgreSQL 버전   지원 중인 버전 표에서 선택
  설치 모드         single / ha  (ha 면 etcd, Patroni 패키지 추가)
  pgmon 포함        Y 권장
  의존 rpm 다운로드 Y  (CRB/PowerTools 자동 활성화, sudo 필요)
  pg_tune           항상 포함 (tools/pg_tune)

결과 확인
  postgresql/BUILD_LOCK.txt          받은 구성 기록
  postgresql/VERIFY_REPORT.txt       (--verify 시) 빌드 검증 결과
  postgresql/installer/build_features.txt   이 PG 버전에서 켤 configure 옵션
  postgresql/PKG/                    rpm 들

압축
$ tar -czf postgresql.tar.gz postgresql/
```

5. [2단계] 폐쇄망 반입

```
single : DB 서버로 복사
HA     : etcd 서버(설치 실행 서버)로만 복사. 나머지 노드는 설치 스크립트가 배포

$ tar -xzf postgresql.tar.gz
$ cd postgresql
```

6. [3단계] single 설치

```
$ bash postgresql_install.sh
  설치 모드: 1) single
```

```
입력 항목 (괄호는 기본값)
  CPU core 수 / Memory(GB)   (현재 서버 값)   -> postgresql.conf 튜닝 기준
  PostgreSQL Port            (5432)
  엔진 설치 경로              (/postgres)      -> /postgres/app
  Data 영역 경로              (/pgdata)        -> /pgdata/data, /pgdata/log
  DB Locale                  (ko_KR.utf8)     -> 나중에 변경 불가 (재구축 필요)
  백업 선택                   (basebackup)  pgbackrest / basebackup
  [basebackup]
  Archive 사용               (Y)  상위 경로 (/pgbackup) -> /pgbackup/arc
  Backup 사용                (Y)  상위 경로 (/pgbackup) -> /pgbackup/back
    백업 주기                1) 매일 02:00  2) 매주 일 02:00  3) 직접(cron)
    백업 유지 개수           (7)
    복제 유저명              (replicator)
  [pgbackrest]
  Archive 사용               (Y)  WAL -> archive-push -> 저장소
    저장소 상위 경로         (/pgbackup) -> /pgbackup/pgbackrest
    stanza 이름              (main, HA 는 클러스터 이름)
  Backup 사용                (Y)
    백업 주기                1) 일요일 full + 월~토 diff (권장)  2) 매일 full  3) 직접
    full 보관 개수           (2)

진행
  rpm 설치 -> 소스 빌드 + contrib + extension(PGXS) -> ~/.postgresrc
  -> postgresql.conf 생성 -> initdb(--data-checksums) -> 기동
  -> 복제 유저 / 백업 스크립트 / 크론 -> pg_stat_statements 생성
  pgbackrest 면 추가로: pgBackRest 빌드 -> stanza-create -> check
                        -> 첫 full 백업 -> 크론

완료 후
$ source ~/.bashrc
$ psql -c "select version();"
```

7. [3단계] HA 설치

```
etcd 서버에서 실행
$ bash postgresql_install.sh
  설치 모드: 2) ha

입력 항목 (순서대로)
  클러스터 이름              (pg-cluster)
  etcd 서버                  현재 서버 호스트명
  Primary / Standby          호스트명 (/etc/hosts 에 등록된 이름)
  SSH 포트                   (22)
  etcd 구성                  1) 단일 노드   2) 3노드 (권장)
  VIP 사용                   (N)  y 면 -> VIP 주소 / prefix(24) / 인터페이스(자동)
  watchdog                   1) automatic (권장)  2) required  3) off
  failsafe_mode              (Y)
  synchronous_mode           (N)  - 데이터 유실 0 이 필요하면 y (커밋 지연 증가)
  PostgreSQL Port            (5432)
  복제 유저명 / 패스워드
  Primary / Standby CPU, Memory
  엔진 / Data / Locale / Archive / Backup   (single 과 동일)
  백업 서버                  Primary 또는 Standby (기본: Standby)   (basebackup)
  pgbackrest 를 고르면       저장소 = etcd 서버 (이 서버), 백업은 etcd 서버
                             크론이 Standby 에서 복사. 백업 서버 질문 없음

확인 화면에서 etcd / VIP / 안전장치 값 확인 후 Y

진행 단계
  pre-flight  SSH, sudo, /etc/hosts, VIP 도구(ip/arping/curl) 점검
              하나라도 실패하면 조치 방법 출력 후 중단
  PHASE 1     etcd 설치 + systemd 등록 (1노드 / 3노드)
```

```
  PHASE 2     etcd 서버에서 PG + extension 빌드
  PHASE 3     Primary: 엔진 배포, rpm, .postgresrc, patroni.yml, patroni.service
  PHASE 4     Standby: 엔진(Primary -> Standby), 나머지 동일
  PHASE 5     VIP 스크립트 배포 + 보정 cron
              watchdog 준비 (softdog 모듈, /dev/watchdog 권한)
              Patroni 설치 -> Primary 기동 -> Standby 기동(자동 클론)
              patronictl list, 백업 / WAL 도구 배포
              VIP 위치 확인, vip_watch 배포 + 1회 실행 결과 출력
  PHASE 6     etcd 서버 빌드 결과물 삭제
              (pgbackrest 면 etc/backup/pgbackrest 만 남김 = 저장소용)
              pgbackrest: DB 노드 -> etcd SSH 키 등록, 설정 배포,
              stanza-create -> check -> 첫 full 백업 -> 크론 (etc 서버)

※ etcd 서버에 이미 single PostgreSQL (pgmon 저장소 DB) 이 같은 엔진 경로에
  있으면, PHASE 2 전에 /postgres -> /postgres.ha_keep_<시각> 로 옮겨두고
  (실행 중이면 중지) PHASE 6 에서 되돌린 뒤 다시 기동한다. 실패해도 되돌림.

완료 화면에 etcd / Primary / Standby / VIP / 감시자 경로가 표시됨
```

8. 무인 설치 (응답 파일)

```
$ cp install.conf.sample install.conf && chmod 600 install.conf
$ vi install.conf
$ bash postgresql_install.sh -c install.conf      # 빠진 항목만 질문
$ bash postgresql_install.sh -c install.conf -y   # 완전 무인 (빠진 항목 = 기본값)

HA 예시 (값은 환경에 맞게)
  MODE_SEL=2
  CLUSTER_NAME=pg-cluster
  HA_ETCD_HOST=pgetcd
  HA_PRIMARY_HOST=pgdb1
  HA_STANDBY_HOST=pgdb2
  ETCD_MODE_SEL=2
  HA_VIP_USE=y
  HA_VIP=10.0.0.100
  HA_VIP_PREFIX=24
  HA_WATCHDOG_SEL=1
  HA_FAILSAFE=y
  HA_SYNC_MODE=n
  REPL_PASS_HA=<복제 패스워드>
나머지 키는 install.conf.sample 주석 참고
```

9. 설치 후 확인

```
[single]
 [ ] psql -c "select version();"
 [ ] psql -c "show shared_preload_libraries;"        -> pg_stat_statements
 [ ] psql -c "select * from pg_stat_archiver;"       -> failed_count = 0
 [ ] crontab -l                                      -> pg_basebackup, wal_cleanup
 [ ] bash /postgres/app/etc/backup/basebackup/pg_basebackup.sh   (1회 수동 백업)
 [ ] bash /postgres/app/etc/backup/pitr_restore.sh --list
 (pgbackrest 면 위 백업 2줄 대신)
 [ ] pgbackrest --stanza=main info        -> status ok, full backup 1개
 [ ] crontab -l                           -> pgbackrest full / diff

[HA]  (Primary 에서)
 [ ] patronictl -c /postgres/app/etc/patroni/patroni.yml list
       Leader 1개, Replica 1개, State=streaming, Lag 0
 [ ] patronictl -c /postgres/app/etc/patroni/patroni.yml show-config
       failsafe_mode / synchronous_mode 값 확인
 [ ] etcdctl --endpoints=http://<etcd>:2379 endpoint health      (3노드면 3개)
 [ ] ip -o -4 addr show | grep pgvip         -> Primary 에만 VIP
 [ ] ls -l /dev/watchdog                     -> 실행 계정 소유 (watchdog 사용 시)
```

```
 [ ] journalctl -u patroni | grep -i watchdog
 [ ] crontab -l                              -> vip_check, wal_cleanup
                                                (백업 서버는 pg_basebackup 도)
(etcd 서버에서)
 [ ] bash ~/pgvip/vip_watch.sh --once        -> "OK 리더=[primary] VIP보유=[primary]"
 [ ] pgbackrest --stanza=pg-cluster info     -> status ok, full backup 1개  (pgbackrest)
 [ ] pgbackrest --stanza=pg-cluster check    -> Primary archive-push 정상    (pgbackrest)
 [ ] crontab -l                              -> pgbackrest full / diff     (pgbackrest)
 [ ] crontab -l                              -> vip_watch
 [ ] 앱 서버에서 psql -h <VIP> -c "select pg_is_in_recovery();"  -> f

[보안 - 운영 전 필수]
 [ ] single: postgres 패스워드 설정, pg_hba.conf 의 local trust 조정
 [ ] HA: patroni.yml superuser 패스워드(기본 postgres) 변경, /24 trust 범위 검토
 [ ] pgmon role 패스워드(기본 = role 이름) 변경
```

10. HA 동작 테스트 시나리오

```
자동 : etcd 서버(postgres 계정)에서  bash ha_test.sh
       T0 사전 상태 / T1 쓰기·복제 / T2 백업 / T3 switchover / T4 Patroni 정지
       / T5 VIP 보정 / T6 etcd 정지 를 순서대로 실행하고 ~/ha_test_report_<시각>.txt 생성
       T7 watchdog(노드 재부팅) 은  bash ha_test.sh --only T7 --with-watchdog
       전원 OFF 는 아래 T3 처럼 수동

수동 준비 : 터미널 3개
  (A) etcd 서버  : watch -n 2 "bash ~/pgvip/vip_watch.sh --once"
  (B) Primary    : tail -f /pgdata/log/vip.log
  (C) 앱/클라이언트 :
        while true; do
          psql -h <VIP> -Atc "select pg_postmaster_start_time(), pg_is_in_recovery(), now()"
          sleep 1
        done
        (postmaster 기동 시각이 바뀌면 다른 노드로 붙은 것)

T1. switchover (계획 전환)
  $ patronictl -c /postgres/app/etc/patroni/patroni.yml switchover pg-cluster
  기대 : 수 초 내 역할 교대, VIP 가 새 Primary 로 이동, (C) 의 기동 시각 변경
         vip_watch 에 LEADER_CHANGED 알림

T2. Patroni 정상 정지 (Primary)
  $ sudo systemctl stop patroni
  기대 : PG 도 같이 정지, Standby 승격, VIP 이동. 정상 정지라 watchdog 리부트 없음
  복구 : sudo systemctl start patroni  -> Replica 로 재합류 (pg_rewind)

T3. Primary 서버 강제 종료 (전원 OFF / VM 강제 정지)
  기대 : 약 30~40초 후 Standby 승격 + VIP 이동
  복구 : 서버 기동 -> Patroni 가 Replica 로 재합류

T4. 펜싱(watchdog) 테스트  ※ 노드가 리부트됨
  Primary 에서 Patroni 를 강제로 멈춤
  $ sudo kill -STOP $(systemctl show -p MainPID --value patroni)
  기대 : 약 25초 뒤 softdog 이 Primary 노드 리부트
         -> Standby 승격 + VIP 이동, 멈춘 노드가 Primary 로 살아남지 않음
  watchdog=off 로 설치했다면 이 테스트에서 리부트가 일어나지 않음 (그게 위험 상황)

T5. Primary 네트워크 단절
  Primary 콘솔(서버 직접 접속)에서 서비스 인터페이스 down 또는 iptables 로 차단
  기대 : Primary 가 스스로 demote + VIP 제거, Standby 승격 + VIP 추가
  복구 후 : vip_watch 에서 VIP 중복 없음 확인

T6. etcd 서버 정지
  $ sudo systemctl stop etcd          (etcd 서버)
  3노드 : 영향 없음 (quorum 유지)
```

```
  1노드 : failsafe_mode=true 면 Primary 유지, false 면 Primary demote(쓰기 중단)
  VIP : DB 노드 쪽 관리 계속, vip_watch 만 멈춤

T7. callback 누락 보정
  Primary 에서 VIP 를 수동 제거
  $ sudo ip addr del <VIP>/24 dev <인터페이스>
  기대 : 20~40초 안에 vip_check 가 다시 붙임 (vip.log), 또는 vip_watch 가 VIP_FIXED 알림

각 테스트 후 확인
  patronictl list / vip_watch.sh --once / pgmon 복제 화면
```

11. 설치 문제 해결

증상	원인 / 조치
pre-flight SSH 실패	키 교환 방향 확인 ssh -o BatchMode=yes <호스트> 로 직접 테스트
pre-flight sudo 실패	/etc/sudoers.d/<계정> NOPASSWD, 권한 440
/etc/hosts 항목 없음	세 서버 모두 같은 내용으로 등록
configure 에서 옵션 빠짐 경고	해당 -devel rpm 이 PKG/ 에 없음 -> 다운로드 서버에서 --verify 로 확인, 필요 없으면 무시
extension 빌드 실패	설치는 계속됨. 실패 목록 확인 후 수동 빌드
locale 오류로 initdb 실패	locale -a 로 존재 확인, glibc-langpack-ko 설치
Patroni 기동 실패	journalctl -u patroni -n 100 etcd 접속(2379), patroni.yml 들여쓰기 확인
Standby 가 클론 안 됨	Primary <-> Standby 5432 방화벽, 복제 유저 / pg_hba
VIP 가 안 붙음	sudo -n ip addr 동작 확인, /pgdata/log/vip.log VIP 인터페이스 자동 선택 실패 시 HA_VIP_IFACE 지정
VIP 는 붙었는데 접속 안 됨	스위치 ARP 캐시 -> arping 수동: sudo arping -U -c 3 -I <dev> <VIP>
watchdog 경고 (사용 불가)	lsmod | grep softdog, ls -l /dev/watchdog 다른 프로세스 점유 여부 (sudo lsof /dev/watchdog)
설치 로그	/pgdata/log/pg_install.log HA 는 /pgdata/log/ha_setup.log 도

END